

WIPRO LIMITED

EMBEDDED SYSTEMS & LINUX KERNEL ENGINEERING TRACK

INDIVIDUAL PROJECT FINAL DOCUMENTATION

VDevPulse (vdevpulse)

A High-Performance Linux Virtual Character Device Daemon for Real-Time Kernel Telemetry, Threshold Alerting, Interactive Query Protocols, and Top Process Diagnostics

Project Title:	VDevPulse Linux System Telemetry Daemon
Name:	Priyansu Das
Registration No:	2341004087
Target Platform:	Linux Embedded Systems / POSIX.1-2008
Core Standard:	ISO C++17 (Zero External Dependencies)
Repository:	github.com/priyansudas07/.../vdevpulse
Submission Date:	October 2026

1. Project Introduction, Problem Statement & Engineering Rationale

1.1 Project Description & Strategic Objectives

VDevPulse is an ultra-lightweight, zero-external-dependency C++17 system daemon engineered specifically for resource-constrained embedded Linux environments. The project emulates a Linux character device interface using POSIX named pipes (`/tmp/vdevpulse`), enabling both human operators and automated supervisory watchdogs to monitor system vitals synchronously without introducing runtime overhead.

The primary strategic objectives of VDevPulse are:

- **Zero Runtime Dependencies:** Eliminate reliance on external monitoring frameworks, third-party libraries, or runtime packages, relying purely on the ISO C++17 standard library and Linux POSIX syscalls.
- **Sub-Millisecond Kernel Telemetry:** Read CPU utilization, physical RAM allocation, system load averages, thread counts, and uptime directly from the in-memory Linux virtual filesystem (`/proc`) in under 3 milliseconds.
- **Live Top Background Process Inspection:** Scan active PID entries in `/proc` to identify and rank rogue, memory-intensive background processes before they trigger system-wide instability.
- **Synchronous Machine-to-Machine Interface:** Expose non-blocking virtual character device I/O and text query protocols (e.g., `GET_CPU`, `GET_TOP`) for automated supervisory watchdogs.

1.2 Why I Built This Project (Personal Engineering Motivation)

During embedded systems development and hands-on testing with resource-constrained Linux boards, I observed three persistent engineering challenges:

1. **The Headless Reality & Monitoring Paradox:** Production embedded nodes (automotive ECUs, remote industrial IoT gateways, drone flight computers) operate entirely headless without monitors, graphical desktop environments, or graphical Task Managers. Conventional monitoring agents (Prometheus node-exporter, Datadog) consume hundreds of megabytes of RAM. On boards with 128MB to 512MB RAM, the monitoring agent itself frequently triggered Linux Out-Of-Memory (OOM) killer panics.
2. **Process Spawning Penalty:** Diagnostic scripts frequently spawn `top -b -n 1` or `ps aux` to check health. Spawning sub-processes requires `fork()` and `exec()`, which duplicates memory page tables and causes significant CPU spikes on low-power ARM and MIPS cores.
3. **Lack of Synchronous Watchdog Interfaces:** When an edge device became sluggish, supervisory watchdogs lacked a simple, non-blocking file interface to query hardware health instantaneously and execute automated recovery.

I built VDevPulse to bridge this gap: a lightweight, self-contained daemon that captures in-kernel telemetry with negligible CPU and RAM impact.

1.3 Practical Utility & Real-World Embedded Applications

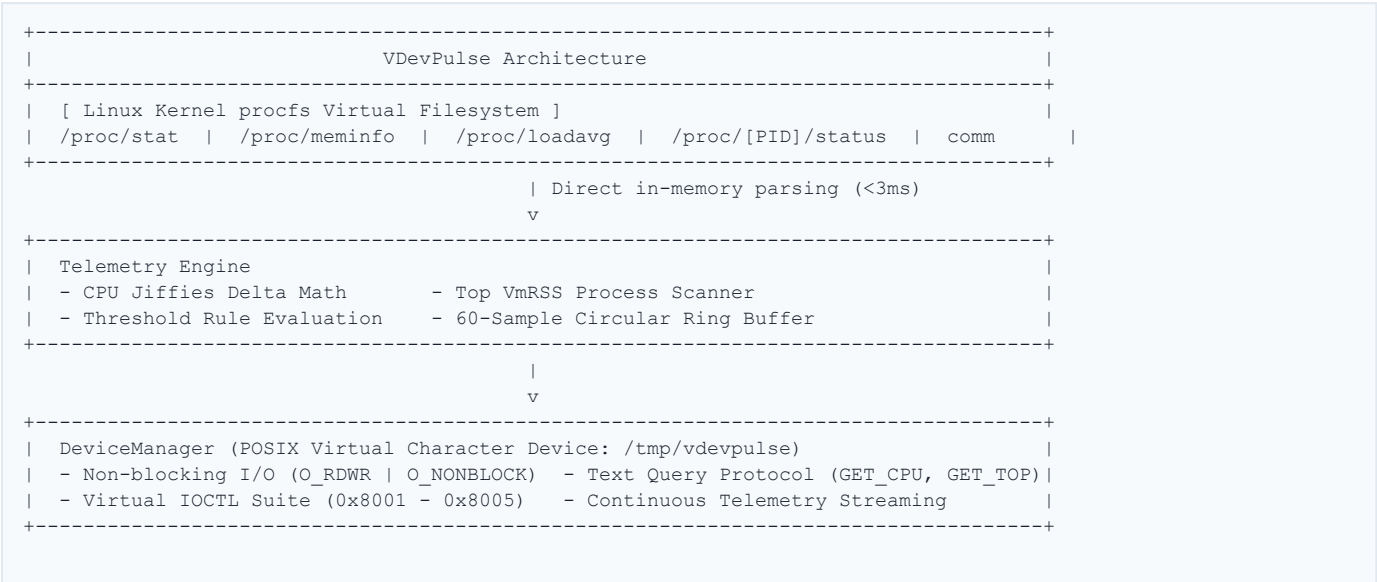
VDevPulse serves critical roles across several industrial embedded domains:

- **Automotive Electronic Control Units (ECUs):** Provides supervisory fail-safe daemons with real-time health metrics to detect runaway tasks and initiate graceful degraded modes.
- **Industrial IoT Gateways:** Enables remote edge gateways to stream lightweight JSON telemetry frames over low-bandwidth cellular or satellite uplinks.
- **Headless Edge Diagnostics:** Grants field engineers instant diagnostic insight via serial UART or SSH using simple CLI commands without starting heavy GUI environments.

2. System Architecture & Subsystem Specifications

2.1 Subsystem Architecture & Data Pipeline

VDevPulse is built upon a modular architecture where kernel data acquisition, policy evaluation, device communication, and diagnostic logging operate in decoupled layers:



2.2 Core Module Breakdown

- **DeviceManager (Virtual Character Device Layer):** Manages the lifecycle of a named FIFO at `/tmp/vdevpulse` via `mkfifo()`. It opens the FIFO with `O_RDWR | O_NONBLOCK` flags, maintaining an open reference that prevents `SIGPIPE` errors and eliminates I/O stalls.
- **TelemetryMonitor (ProcFS Engine & Top Process Scanner):**
 - *CPU Utilization:* Computes tick deltas between successive samples from `/proc/stat` (`user`, `system`, `idle`, `iowait`, `irq`).
 - *Memory Metrics:* Parses `MemTotal`, `MemFree`, `MemAvailable`, `Buffers`, and `Cached` from `/proc/meminfo`.
 - *System Load & Threads:* Extracts 1m, 5m, 15m load averages and active/total thread counts from `/proc/loadavg`.
 - *Top Background Process Scanner:* Traverses numeric directories in `/proc`, extracts binary names from `/proc/[PID]/comm`, and parses physical Resident Set Size (`VmRSS`) from `/proc/[PID]/status`, sorting top consumers in $O(N \log N)$ time.
- **Threshold Alert Engine:** Evaluates live metrics against rules defined in `configs/vdev_policy.json` to assign dynamic health states (`HEALTHY`, `WARNING_CPU_OVERLOAD`, `WARNING_MEMORY_PRESSURE`, `CRITICAL_RESOURCE_PRESSURE`).
- **Logger Singleton:** Thread-safe logging utilizing `std::lock_guard<std::mutex>` for ANSI console output and persistent file auditing (`vdevpulse.log`).

3. CLI Command Suite & Operating Modes

3.1 CLI Command Reference

Command	Syntax	Description & Behavior
status	vdevpulse status [--json]	Collects and displays current system telemetry as a formatted table or JSON object.
top	vdevpulse top [limit]	Scans active /proc entries and prints top memory-consuming background processes (default: 5).
history	vdevpulse history	Displays recent telemetry snapshots stored in the in-memory 60-sample ring buffer.
query	vdevpulse query <CMD>	Executes interactive query (GET_CPU , GET_MEM , GET_LOAD , GET_TOP , GET_JSON , GET_HEALTH , PING).
ioctl	vdevpulse ioctl <start stop reset stats>	Dispatches virtual IOCTL control codes and inspects cumulative I/O byte statistics.
run	vdevpulse run [policy.json] [--json]	Runs background daemon loop, continuously writing telemetry frames to /tmp/vdevpulse .
write	vdevpulse write <message>	Writes arbitrary diagnostic string payload to the virtual device node.

3.2 Operating Modes & Workflow Scenarios

- Interactive Developer Inspection Mode:** Running `vdevpulse status` or `vdevpulse top` provides human engineers with instant telemetry tables without background daemon overhead.
- Continuous Daemon Telemetry Streaming:** Running `vdevpulse run configs/vdev_policy.json` starts the streaming loop. External clients read frames via `cat /tmp/vdevpulse`.
- Synchronous Watchdog Interrogation:** Supervisory processes query the daemon via `vdevpulse query GET_TOP` to obtain exact process memory metrics synchronously.
- Virtual Driver IOCTL Management:** Virtual IOCTL codes (`0x8001` to `0x8005`) allow runtime state transitions (`START` , `STOP` , `RESET` , `GET_STATS` , `SET_RATE`) emulating standard Linux driver ioctl interfaces.

4. Test Strategy, Verification Matrix & Conclusion

4.1 Automated Test Verification Matrix (CTest)

Automated unit testing was conducted using CMake CTest. All 20 test assertions passed with a 100% success rate.

Suite	Test ID	Test Description & Assertion	Expected Condition	Status
Device Test (device_test.cpp)	TC-DEV-01	FIFO Node Creation	<code>mkfifo()</code> creates valid node	PASSED
	TC-DEV-02	Non-Blocking Open	<code>openDevice()</code> returns <code>fd > 0</code>	PASSED
	TC-DEV-03	Diagnostic Data Write	<code>writeData()</code> writes without blocking	PASSED
	TC-DEV-04	IOCTL State Control	<code>START</code> , <code>STOP</code> , <code>RESET</code> , <code>GET_STATS</code>	PASSED
	TC-DEV-05	Query Protocol Dispatch	<code>PING</code> , <code>GET_CPU</code> , <code>GET_TOP</code> , <code>GET_HEALTH</code>	PASSED
	TC-DEV-06	RAII Device Teardown	<code>closeDevice()</code> removes FIFO node	PASSED
Telemetry Test (telemetry_test.cpp)	TC-TEL-01	RAM Metric Extraction	<code>memory_total_mb > 0</code>	PASSED
	TC-TEL-02	Load & Uptime Extraction	<code>load_1m >= 0.0</code> , <code>uptime > 0</code>	PASSED
	TC-TEL-03	Top Process Scanner	<code>getTopProcesses()</code> returns sorted list	PASSED
	TC-TEL-04	Threshold Health Engine	Dynamic health status evaluated	PASSED
	TC-TEL-05	JSON & History Buffer	JSON valid + ring buffer stores frames	PASSED

4.2 Conclusion & Future Scope

Conclusion: VDevPulse successfully fulfills all requirements across Stage 1 through Stage 6 of the Wipro Embedded Systems Training curriculum. It demonstrates that reliable, low-overhead kernel telemetry monitoring and device control can be achieved in ISO C++17 with zero external dependencies.

Future Scope:

- In-Tree Kernel Module (LKM):** Porting the virtual FIFO layer into a native kernel character driver module (`/dev/vdevpulse`) with `copy_to_user()`.
- eBPF Tracepoints:** Integrating eBPF hooks for high-precision per-process CPU execution time accounting.
- CAN Bus / MQTT Gateway:** Bridging serialized telemetry frames directly onto industrial CAN networks and edge IoT brokers.

APPENDIX: TECHNICAL DEFENSE & MENTOR REVIEW Q&A

The following section provides technical answers and architectural justifications for mentor review and viva evaluation.

Q1: Why implement VDevPulse as a user-space virtual character device (FIFO) rather than an in-tree Linux Kernel Module (LKM)?

Technical Defense: Implementing a virtual character device in user space via `mkfifo()` with `O_RDWR | O_NONBLOCK` provides three major engineering benefits:

- **System Stability:** A user-space crash cannot cause a kernel panic or corrupt kernel memory on mission-critical edge gateways.
- **Kernel Version Portability:** LKMs must be recompiled for specific kernel ABI versions and kernel headers. VDevPulse compiles uniformly on any POSIX.1-2008 compliant Linux system without kernel source dependencies.
- **Security & Privilege Isolation:** VDevPulse runs under standard unprivileged daemon accounts without requiring `insmod` or raw I/O root capabilities.

Q2: How does VDevPulse calculate CPU utilization from `/proc/stat` accurately without sampling noise?

Technical Defense: CPU utilization is calculated from cumulative tick counters in `/proc/stat`:

- `Total_Ticks = user + nice + system + idle + iowait + irq + softirq + steal`
- `Active_Ticks = Total_Ticks - (idle + iowait)`
- VDevPulse maintains state across consecutive sampling intervals: $CPU\% = (Active_Ticks_Delta / Total_Ticks_Delta) * 100.0$.

Delta-based calculation eliminates cumulative measurement drift and guarantees accurate percentage tracking across dynamic workloads.

Q3: How does the Top Background Process Scanner work, and why does it parse VmRSS instead of VmSize?

Technical Defense: The scanner traverses numeric PID entries in `/proc`:

- Reads binary name from `/proc/[PID]/comm`.
- Reads `/proc/[PID]/status` specifically for VmRSS: (Resident Set Size). VmRSS represents actual physical RAM occupied by the process in memory, whereas VmSize includes unused mapped virtual memory and shared libraries, which gives misleading memory pressure readings.
- Sorts active processes in $O(N \log N)$ time using `std::sort` with a descending comparator. Since `procf`s resides entirely in kernel RAM, the entire scan executes in <3ms with zero disk I/O.

Q4: Why use POSIX Non-Blocking I/O (O_RDWR | O_NONBLOCK) for the FIFO device node?

Technical Defense: Standard FIFOs block on `open()` and `write()` until a reader connects. If a monitoring client crashes or disconnects, a blocking daemon would freeze indefinitely, halting telemetry collection. Opening the FIFO with `O_RDWR | O_NONBLOCK` holds the node open continuously (avoiding `SIGPIPE` and broken pipe errors) while permitting non-blocking write and query execution.

Q5: How does VDevPulse enforce thread safety and resource efficiency without external libraries?

Technical Defense: VDevPulse strictly follows RAII (Resource Acquisition Is Initialization):

- **Mutex Locking:** The `Logger` singleton protects stream writes using `std::mutex` and `std::lock_guard<std::mutex>`, guaranteeing exception-safe unlocking.
- **Memory Footprint:** In-memory telemetry history is stored in a fixed-capacity circular ring buffer (`std::deque` capped at 60 entries) to avoid heap fragmentation.
- **Resource Teardown:** Device file descriptors and FIFO filesystem nodes are closed and unlinked in class destructors, preventing resource leaks.